Customer / Client / Freelancer / Admin / Expert
:Next.js Frontend
:AuthController
:AuthService
:TokenService
:Google OAuth2
:AuthRepository
:SharedRole Repository
:PrismaService
Database
1. Click "Đăng nhập với Google"
2. GET /auth/google-link
3. getAuthorizationUrl({userAgent, ip})
4. encode state = base64url({userAgent, ip})
5. generateAuthUrl(scopes, state, access_type="offline")
6. authorizationUrl
7. return { url }
8. return { url }
9. Redirect browser → authorizationUrl
10. Chọn tài khoản Google + Cho phép
11. 302 Redirect → /auth/google/callback? code=...&state=...
12. GET /auth/google/callback? code=...&state=...
13. googleCallback({code, state})
14. decode state → {userAgent, ip}
15. getToken(code)
16. { access_token, refresh_token }
17. GET /oauth2/v2/userinfo
18. { id, email, name, picture }
19. Check email
opt
[email is null]
19.1. throw EmailNotFoundException
19.2. 302 Redirect → NEXT_URL/login?error=google
19.3. GET /login?error=google
19.4. Show message "Email not found"
20. findUserByEmail IncludeRoles(email)
21. user.findUnique({where: {email}, include: {userRoles: true}})
22. Execute query
23. Return result
24. User | null
25. User | null
26. Check user exists
opt
[user is null - new registration]
26.1.1. getClientRoleId()
26.1.2. role.findUnique({where: {name: "CLIENT"}})
26.1.3. Execute query
26.1.4. Return result
26.1.5. Return role
26.1.6. clientRoleId
26.1.7. Check client role result
opt
[get client role failed]
26.1.8. Return error
26.1.9. 302 Redirect → NEXT_URL/login?error=google
26.1.10. GET /login?error=google
26.1.11. Show message "Login with Google failed"
26.2.1. createUserAnd RegisterGoogle(email, fullName, roleId)
26.2.2. user.create({data: {..., userRoles: {create: {roleId}}}})
26.2.3. Execute query (transaction)
26.2.4. Return result
26.2.5. newUser (with userRoles)
26.2.6. newUser
26.2.7. Check create user result
opt
[create user failed]
26.2.8. Return error
26.2.9. 302 Redirect → NEXT_URL/login?error=google
26.2.10. GET /login?error=google
26.2.11. Show message "Login with Google failed"
26.3.1. createOauthAccount(userId, GOOGLE, providerUserId)
26.3.2. oauthAccount.create({data})
26.3.3. Execute query
26.3.4. Return result
26.3.5. Return account
26.3.6. Return account
26.3.7. Check create oauth account result
opt
[create oauth account failed]
26.3.8. Return error
26.3.9. 302 Redirect → NEXT_URL/login?error=google
26.3.10. GET /login?error=google
26.3.11. Show message "Login with Google failed"
27. Check user status
opt
[user.isBanned = true]
27.1. throw UserBannedException
27.2. 302 Redirect → NEXT_URL/login?error=google
27.3. GET /login?error=google
27.4. Show message "Account is banned"
28. signRefreshToken({ userId })
29. refreshToken (JWT)
30. createSession({userId, refreshToken, userAgent, ip, expiresAt})
31. session.create({data})
32. Execute query
33. Return result
34. session { id }
35. session
36. Check create session result
opt
[create session failed]
36.1. Return error
36.2. 302 Redirect → NEXT_URL/login?error=google
36.3. GET /login?error=google
36.4. Show message "Login with Google failed"
37. signAccessToken({userId, roleId, roleName, sessionId})
38. accessToken (JWT)
39. { accessToken, refreshToken }
40. 302 Redirect → NEXT_URL/api/auth/google? accessToken=...&refreshToken=...
41. GET /api/auth/google? accessToken=...&refreshToken=...
42. Save tokens (cookie / localStorage)
43. Redirect to home page